

5주차 : 딥러닝 파이토치 교과서 6장- Part 1(~6.1.4)

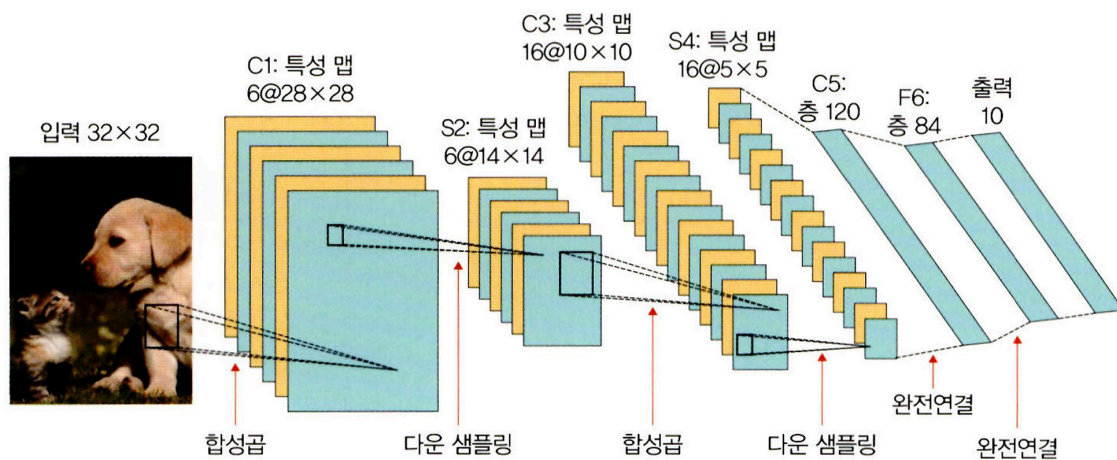
6장. 합성곱 신경망 2

6.1 : 이미지 분류를 위한 신경망

- 입력 데이터로 이미지를 사용한 분류 : 특정 대상이 영상 내에 존재하는지 여부를 판단

6.1.1 : LeNet-5

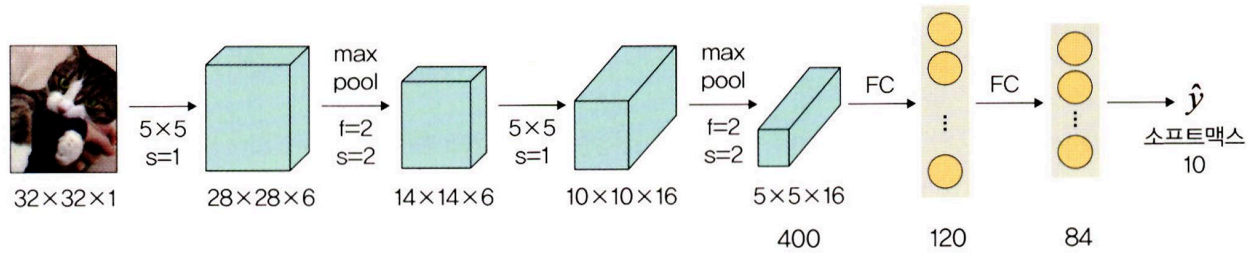
▼ 그림 6-1 LeNet-5



- LeNet - 5 : 합성곱(convolutional)과 다운샘플링(sub-sampling) 혹은 풀링을 반복적으로 거치면서 마지막에 완전연결층에서 분류를 수행함.
 - C : 합성곱층, S : 풀링층, F : 완전연결층

LeNet - 5 예제 살펴보기

▼ 그림 6-2 LeNet-5 예제 신경망



▼ 표 6-1 LeNet-5 예제 신경망 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	—	—	—
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	—
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	—
완전연결층	—	120	—	—	렐루(ReLU)
완전연결층	—	84	—	—	렐루(ReLU)
완전연결층	—	2	—	—	소프트맥스(softmax)

코드 6-1 : 필요한 라이브러리 호출

```
!pip install --user tqdm
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms # 이미지 변환(전처리) 기능을
제공하는 라이브러리
from torch.autograd import Variable
from torch import optim # 경사하강법을 활용하여 가중치를 구하기 위
한 옵티마이저 라이브러리
import torch.nn as nn
import torch.nn.functional as F
import os # 파일 경로에 대한 함수들을 제공
```

```

import cv2
from PIL import Image
from tqdm import tqdm_notebook as tqdm # 진행 상황을 가시적으로
표현 - 모델의 학습 경과를 확인하고 싶을 때 사용하는 라이브러리
import random
from matplotlib import pyplot as plt

device = torch.device("cuda:0" if torch.cuda.is_available() e
lse "cpu")

```

코드 6-2 : 이미지 데이터셋 전처리

```

class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=
(0.5, 1.0)), # 입력 이미지를 주어진 크기로 조정
                transforms.RandomHorizontalFlip(), # 주어진 확
률로 이미지를 수평 반전
                transforms.ToTensor(), # torch.FloatTensor 배
열로 바꾸어 주는 메서드
                transforms.Normalize(mean, std) # 입력 데이터
정규화 : 데이터의 스케일을 줄이고 0 근처 분포로 만들어 학습을 잘 되게
도움
            ]),
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }

```

```
def __call__(self, img, phase):
    return self.data_transform[phase](img) # 클래스를 호출
할 수 있도록 함
```

- 정규화 : 기본적으로 학습할 때 정규화를 진행 → 모델 가중치 업데이트가 안정적으로 이루어지도록 함
 - ImageNet 기반 사전 학습 모델을 가져와 전이 학습을 할 때는 그에 맞는 수치를 (mean, std)를 똑같이 써야 기존 학습 결과를 그대로 활용할 수 있음
 - 추후 코드 6-7에서 변수 값을 정의해 주게 됨 (mean, std + a)
- **train (학습용):**
 - **RandomResizedCrop** : 이미지를 무작위 위치에서 크롭하고 리사이즈합니다.
 - **RandomHorizontalFlip** : 이미지를 무작위로 좌우 반전시킵니다.
 - **목적:** 모델이 이미지의 특정 위치나 방향에 과적합(Overfitting)되지 않도록, 매 에포크마다 조금씩 변형된 이미지를 보여주어 **일반화 성능**을 높입니다.
- **val (검증용):**
 - **Resize(256)** 후 **CenterCrop(resize)** : 무작위 변형 없이 이미지 중앙을 정확하게 잘라내어 입력 규격(resize)을 맞춥니다.
 - **목적:** 모델 성능 측정 시 **항상 동일하고 공정한 기준**으로 평가하기 위함입니다. 검증할 때 원본 이미지를 무작위로 자르거나 뒤집으면 평가 결과가 매번 달라지므로, 일관된 중앙 크롭만 적용합니다.

코드 6-3 : 이미지 데이터셋을 불러온 후 훈련, 검증, 테스트로 분리

```
cat_directory = 'dogs-vs-cats/Cat/'
dog_directory = 'dogs-vs-cats/Dog/'

cat_images_filepaths = sorted([os.path.join(cat_directory, f)
for f in os.listdir(cat_directory)])
# sorted : 데이터를 정렬된 리스트로 만들어서 반환
# os.path.join : 경로와 파일명을 결합 or 분할된 경로를 하나로 합치고
싶을 때 사용
# os.listdir : 지정한 디렉터리 내 모든 파일의 리스트 반환
```

```

dog_images_filepaths = sorted([os.path.join(dog_directory, f)
for f in os.listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]

random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400] # 훈련용 400개 이미지
val_images_filepaths = correct_images_filepaths[400:-10] # 검증용 92개 이미지
test_images_filepaths = correct_images_filepaths[-10:] # 테스트용 10개 이미지
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))

```

- 경로를 합치는 이유 : cat_directory에 폴더 경로가 들어 있는 것은 맞지만, 그 폴더 안의 개별 사진 파일들의 이름에는 폴더 경로가 붙어있지 않기 때문
 - `os.listdir(cat_directory)` 만 쓸 때: ['0.jpg', '1.jpg', '2.jpg'] (파일 이름만 있음, 접근 불가)
 - `os.path.join` 을 함께 사용한 위 코드의 결과: ['dogs-vs-cats/Cat/0.jpg', 'dogs-vs-cats/Cat/1.jpg', 'dogs-vs-cats/Cat/2.jpg'] (전체 경로 완성, 접근 가능)
- 특정 시드 값을 부여 → 동일한 난수 생성
 - 무작위로 섞는 이유: 특정 클래스나 이미지 타입이 한쪽 데이터셋에 쏠리는 데이터 편향(Bias)을 막기 위해서 필수입니다.
 - `seed` 를 고정하는 이유: 무작위로 섞되, 언제 실행해도 똑같은 데이터 분리 결과를 얻어 실험 결과를 비교/재현할 수 있게 하려고 고정합니다.

코드 6-4 : 테스트 데이터셋 이미지 확인 검수

```

def display_image_grid(images_filepaths, predicted_labels=(),

```

```

cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize
=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # 이미
지의 색상 변경 : BGR -> RGB
        true_label = os.path.normpath(image_filepath).split(o
s.sep)[-2] # 경로명 정규화
        predicted_label = predicted_labels[i] if predicted_la
bels else true_label
        color = "green" if true_label == predicted_label else
"red" # 예측과 정답이 동일 - 초록색, 아니면 빨간색
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(predicted_label, color=color)
        ax.ravel()[i].set_axis_off() # 이미지의 축 제거
    plt.tight_layout() # 이미지의 여백 조정
    plt.show()

# 코드 6-5 : 테스트 데이터셋 이미지 출력
display_image_grid(test_images_filepaths)

```



- **지금 상황 (학습 전, 검수 단계):** `display_image_grid(test_images_filepaths)` 형태로 호출했기 때문에 `predicted_labels` 인자에 아무 값도 들어가지 않아 **빈 값(기본값 `()`)** 상태입니다.
 - 따라서 `if predicted_labels` 조건문이 거짓(False)이 되어, `predicted_label` 변수에 `true_label` (정답)이 그대로 대입됩니다.
 - 그 결과 `true_label == predicted_label` 은 당연히 참(True)이 되므로 **모든 글씨가 초록색으로 출력된** 것입니다.
- **나중 상황 (학습 후, 실제 평가 단계):**
나중에 모델이 실제로 예측한 라벨 리스트를 넘겨줄 것입니다.
 - 예: `display_image_grid(test_images_filepaths, predicted_labels=model_preds)`
 - 이때는 모델의 실제 예측값과 폴더 이름에서 추출한 진짜 정답을 비교하여 **맞으면 초록색, 틀리면 빨간색**으로 표시됩니다.

코드 6-6 : 이미지 데이터셋 클래스 정의

```
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'): # 데이터셋의 전처리
        self.file_list = file_list
        self.transform = transform
        self.phase = phase

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path) # img_path 위치에서 이미지 데이터들을 가져옴
        img_transformed = self.transform(img, self.phase) # 이미지에 'train' 전처리를 적용

        label = img_path.split('/')[1].split('.')[0] # / 와
```

. 를 제거, 분리된 값에서 index 0를 가져옴

```
if label == 'dog':
    label = 1
elif label == 'cat':
    label = 0
return img_transformed, label
```

코드 6-7 : 변수 값 정의

```
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

코드 6-8 : 이미지 데이터셋 정의

```
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size, mean, std), phase='train')
# 훈련 데이터에 train_transforms를 적용
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std), phase='val')
# 검증 이미지에 train_transforms를 적용

index = 0 # 첫번째 이미지 지정 (맨 첫번째 데이터 하나만 꺼내옴)
print(train_dataset.__getitem__(index)[0].size()) # 훈련 데이터의 크기 출력
print(train_dataset.__getitem__(index)[1]) # 훈련 데이터의 레이블 출력
```

```
... torch.Size([3, 224, 224])
0
```

- `train_dataset.__getitem__(0)[0]`: 첫 번째 이미지의 전처리된 텐서 데이터
→ 크기가 `[3, 224, 224]` (3채널, 224x224 크기)

- `train_dataset.__getitem__(0)[1]` : 첫 번째 이미지의 클래스 라벨 숫자 → 0
⇒ '고양이' 라는 이미지가 포함되어 있음을 유추해볼 수 있음

코드 6-9 : 데이터로더 정의

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True) # 앞 코드 6-7에서 설정한 배치 사이즈(32)
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False) # 데이터를 조금씩 쪼개어 가져옴
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader}

# 훈련 데이터셋과 검증 데이터셋을 합쳐서 표현

batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

- 훈련 데이터셋 (`shuffle=True`)인 이유
 - **과적합(Overfitting) 및 편향 방지:** 매 에포크(Epoch)마다 데이터 순서를 무작위로 섞어 주어야 모델이 특정 순서 패턴을 외우지 않고 **이미지의 특징 그 자체**를 학습하게 됩니다.
 - **학습 안정화:** 배치가 항상 다양한 패턴의 데이터로 구성되어 경사하강법이 한쪽으로 쏠리지 않고 안정적으로 수렴합니다.
- 검증 데이터셋 (`shuffle=False`)인 이유
 - **평가 기준의 일관성 (순서 불변):** 검증 과정은 모델의 가중치를 업데이트하는 과정이 아니라, **현재 모델의 실력을 단순 채점(평가)만 하는 단계**입니다.
 - 데이터를 섞든 안 섞든 전체 정확도나 손실(Loss) 결과는 동일합니다.
 - 순서를 고정해두면 매 에포크마다 같은 데이터 순서로 평가되므로, 성능 변화나 특정 예측 결과를 추적하고 비교하기가 훨씬 쉬워집니다.

코드 6-10 : 모델의 네트워크 플러스

```

class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.cnn1 = nn.Conv2d(in_channels=3, out_channels=16,
kernel_size=5, stride=1, padding=0) #2d 합성곱층 적용
        self.relu1 = nn.ReLU() # 활성화 함수
        self.maxpool1 = nn.MaxPool2d(kernel_size=2) # 최대 풀
링 적용
        self.cnn2 = nn.Conv2d(in_channels=16, out_channels=3
2, kernel_size=5, stride=1, padding=0) #2d 합성곱층 적용
        self.relu2 = nn.ReLU() # 활성화 함수
        self.maxpool2 = nn.MaxPool2d(kernel_size=2) # 최대 풀
링 적용
        self.fc1 = nn.Linear(32*53*53, 512) # 완전 연결층
        self.relu5 = nn.ReLU() # 활성화 함수
        self.fc2 = nn.Linear(512, 2) # 완전 연결층 : 최종 출력
노드 수를 2로 설정 - 고양이 / 개 판단
        self.output = nn.Softmax(dim=1) # 최종 출력값을 2개 클래
스에 대한 확률 값으로 변환

    def forward(self, x):
        out = self.cnn1(x)
        out = self.relu1(out)
        out = self.maxpool1(out)
        out = self.cnn2(out)
        out = self.relu2(out)
        out = self.maxpool2(out)
        out = out.view(out.size(0), -1)
        out = self.fc1(out)
        out = self.fc2(out)
        out = self.output(out)
        return out

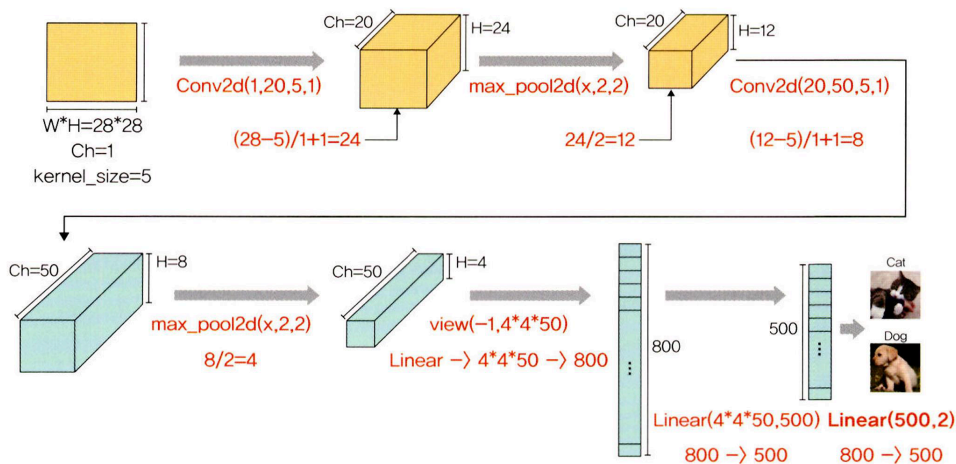
```

코드 6-11 : 모델 객체 생성

```
model = LeNet().to(device)
print(model)
```

- 28×28 크기의 이미지 1장이 연산을 거치며 $50 \times 4 \times 4$ 크기로 축소되고, 이를 1차원으로 펼쳐 총 800개의 특징값을 추출한 뒤 고양이/개(2개)로 최종 분류하는 과정

▼ 그림 6-5 합성곱층과 풀링층의 크기(형태) 계산



코드 6-12 : torchsummary 라이브러리를 이용한 모델의 네트워크 구조 확인

```
!pip install torchsummary
from torchsummary import summary
summary(model, (3, 224, 224))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 220, 220]	1,216
ReLU-2	[-1, 16, 220, 220]	0
MaxPool2d-3	[-1, 16, 110, 110]	0
Conv2d-4	[-1, 32, 106, 106]	12,832
ReLU-5	[-1, 32, 106, 106]	0
MaxPool2d-6	[-1, 32, 53, 53]	0
Linear-7	[-1, 512]	46,023,168
Linear-8	[-1, 2]	1,026
Softmax-9	[-1, 2]	0

=====
 Total params: 46,038,242
 Trainable params: 46,038,242
 Non-trainable params: 0
 =====

Input size (MB): 0.57
 Forward/backward pass size (MB): 19.47
 Params size (MB): 175.62
 Estimated Total Size (MB): 195.67
 =====

코드 6-13 : 학습 가능한 파라미터 수 확인

```
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f'The model has {count_parameters(model):,} trainable parameters')
```

코드 6-14 : 옵티마이저와 손실 함수 정의

```
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9) # 경사 하강법 - 모멘텀 SGD 사용
criterion = nn.CrossEntropyLoss() # 손실함수 정의
```

- 파라미터 설정
 - lr : 학습률
 - momentum : SGD를 가속화하며 흔들림을 줄여 주는 매개변수

```
# 코드 6-15 : 모델의 파라미터와 손실 함수를 CPU에 할당
# GPU를 활성화하지 않은 상태 - CPU 사용
```

```
model = model.to(device)
criterion = criterion.to(device)
```

```
# 코드 6-16 : 모델 학습 함수 정의
```

```
def train_model(model, dataloader_dict, criterion, optimizer,
num_epoch):
    since = time.time()
    best_acc = 0.0

    for epoch in range(num_epoch):
        print('Epoch {}/{}'.format(epoch + 1, num_epoch))
        print('-'*20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train() # 모델을 학습시키겠다는 의미
            else:
                model.eval()

            epoch_loss = 0.0
            epoch_corrects = 0

            for inputs, labels in tqdm(dataloader_dict[phase]):
                inputs = inputs.to(device) # 훈련 데이터셋을 CPU에 할당
                labels = labels.to(device)
                optimizer.zero_grad() # 역전파 실행 전 기울기를 0으로 초기화
```

```

        with torch.set_grad_enabled(phase == 'train'):
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            loss = criterion(outputs, labels) # loss
계산

            if phase == 'train':
                loss.backward() # 학습 가능한 파라미터에
대해 기울기 계산

                optimizer.step() # 파라미터 갱신

            epoch_loss += loss.item() * inputs.size
(0) # 오차와 입력을 곱함
            epoch_corrects += torch.sum(preds == labels.data) # 예측과 정답이 일치하면 epoch_corrects에 저장

            epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset) # 최종 오차 계산
            epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset) # 최종 정확도 계산

            print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase,
epoch_loss, epoch_acc))

            if phase == 'val' and epoch_acc > best_acc: # 검증
데이터셋에 대한 가장 최적의 정확도를 저장
                best_acc = epoch_acc
                best_model_wts = model.state_dict()

            time_elapsed = time.time() - since
            print('Training complete in {:.0f}m {:.0f}s'.format(
                time_elapsed // 60, time_elapsed % 60))
            print('Best val Acc: {:.4f}'.format(best_acc))
            return model

```

- 손실 함수(`criterion`)의 기본 동작 (`reduction='mean'`)
 - PyTorch의 손실 함수(예: `nn.CrossEntropyLoss()`)는 `reduction` 인자의 기본값이 `'mean'` 으로 설정되어 있습니다.
 - 만약 배치 크기(`inputs.size(0)`)가 **32**라면:
 - 손실 함수는 배치 안의 32개 데이터 각각에 대해 오차(Loss)를 계산한 뒤, **32로 나눈 평균값 하나(`loss.item()`)**를 반환합니다.
 - 왜 `loss.item() * inputs.size(0)` 을 곱할까요?
 - 에포크 전체의 평균 손실(`epoch_loss`)을 정확히 구하려면 에포크 전체 데이터의 총 손실 합(Sum)을 구한 뒤, 마지막에 **전체 데이터 개수(`len(dataset)`)로 나누어야** 합니다.
1. `loss.item()` : 배치 내 1개 데이터당 **평균 오차**
 2. `inputs.size(0)` : 현재 배치의 **데이터 개수 (예: 32)**
 3. `loss.item() * inputs.size(0)` : 평균 오차에 데이터 개수를 곱하므로, "해당 배치 전체 데이터 오차의 총합(Sum)"이 됩니다.

코드 6-17 : 모델 학습

```
import time

num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)
```

코드 6-18 : 모델 테스트를 위한 함수 정의

```
import pandas as pd

id_list = []
pred_list = []
_id=0
with torch.no_grad(): # 역전파 중 텐서들에 대한 변화도를 계산할 필요가 없음 -> 훈련 모델과의 가장 큰 차이점
    for test_path in tqdm(test_images_filepaths): # 테스트 데이
```

터셋 이용

```
img = Image.open(test_path)
_id = test_path.split('/')[1].split('.')[1]
transform = ImageTransform(size, mean, std)
img = transform(img, phase='val') # 테스트 데이터셋 전처
```

리 적용

```
img = img.unsqueeze(0) # 텐서에 차원 추가, 0 : 차원이 추
```

가될 위치

```
img = img.to(device)
```

```
model.eval()
```

```
outputs = model(img)
```

```
preds = F.softmax(outputs, dim=1)[:, 1].tolist() # 텐서의 요소가 (0,1) 범위에 있고 합계가 1이 되도록 크기 조정
```

```
id_list.append(_id)
```

```
pred_list.append(preds[0])
```

```
res = pd.DataFrame({
```

```
    'id': id_list,
```

```
    'label': pred_list
```

```
}) # 테스트 데이터셋의 예측 결과인 id와 label을 데이터 프레임에 저장
```

```
res.sort_values(by='id', inplace=True)
```

```
res.reset_index(drop=True, inplace=True)
```

```
res.to_csv('LesNet.csv', index=False) # 데이터 프레임을 csv 파일로 저장
```

- 왜 차원을 추가해야 하나요?

- 테스트 코드에서는 데이터로더(DataLoader)를 거치지 않고 `Image.open()` 으로 이미지를 1장씩 직접 읽어서 전처리했습니다.

- 이 상태 그대로 `model(img)` 에 넘겨주면, 모델은 입력된 텐서의 첫 번째 차원(3)을 '배치 크기(3장)'로 오해하여 연산을 진행하려다가 차원 개수가 맞지 않아 `RuntimeError` (차원 불일치 에러)를 발생시킵니다.

- 따라서 맨 앞에 배치 차원 1 을 덧붙여 주어 "지금 1장짜리 배치 데이터 1묶음이 들어간다"고 알려주는 것입니다.

- `img.unsqueeze(0)` 실행 전: `[3, 224, 224]`
- `img.unsqueeze(0)` 실행 후: `[1, 3, 224, 224]` (0번째 자리에 크기 1인 차원 추가)

코드 6-19 : 테스트 데이터셋의 예측 결과 호출

```
res.head(10)
```

코드 6-20 : 테스트 데이터셋 이미지를 출력하기 위한 함수 정의

```
class_ = classes = {0:'cat', 1:'dog'} # 개와 고양이에 대한 클래스 정의
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]
        if label > 0.5: # 레이블 값이 0.5보다 크다면 개
            label = 1
        else: # 작다면 고양이
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()
```

코드 6-21 : 테스트 데이터셋 예측 결과 이미지 출력

```
display_image_grid(test_images_filepaths)
```

